

Page Replacement

1. Funciones:

bool es_numero(const char *str)

- **Qué hace:** Valida si una cadena de texto (*string*) contiene única y estrictamente dígitos numéricos.
- **Cómo lo hace:** Recorre el string carácter por carácter utilizando la función `isdigit()`. Si encuentra un espacio, una letra o un signo negativo, retorna false. Si termina el recorrido exitosamente, retorna true.
- **Por qué es importante:** Evita que el programa falle (*crash*) si el usuario introduce texto accidentalmente en lugar de números en la consola.

void imprimir_marcos(int frames[], int N)

- **Qué hace:** Imprime de manera visual el estado actual de los marcos de memoria física en la consola (la "traza").
- **Cómo lo hace:** Recorre el arreglo de marcos e imprime su contenido separado por comas dentro de corchetes [...]. Si un marco está vacío (representado con un -1), imprime un guion bajo `_` para denotar ese espacio disponible.

2. Simuladores de algoritmos de reemplazo

Resultado simular_fifo(int N, int secuencia[], int tam, bool mostrar_traza)

- **Qué hace:** Simula el algoritmo **FIFO (First-In, First-Out)**, el cual reemplaza la página que lleva más tiempo en la memoria física.
- **Lógica clave:** Utiliza un índice circular ($\text{indice_insertar} = (\text{indice_insertar} + 1) \% N$). Al no depender de prioridades ni de búsquedas en el futuro, simplemente sobrescribe la posición a la que apunta este índice cada vez que ocurre un fallo de página (*miss*), avanzando al siguiente marco de forma secuencial.
- **Particularidad:** Incluye una bandera `mostrar_traza`. Si es true, imprime el paso a paso en pantalla; si es false, corre en silencio (útil para la demostración de Belády).

Resultado simular_min(int N, int secuencia[], int tam)

- **Qué hace:** Simula el algoritmo **Óptimo (MIN)**, el cual reemplaza la página que tardará más tiempo en volver a ser utilizada en el futuro. Es un algoritmo teórico ideal.
- **Lógica clave:** Cuando la memoria está llena y ocurre un fallo, la función escanea el resto de la secuencia de referencias (desde el paso actual hacia adelante) para cada página presente en los marcos. Identifica cuál aparece más lejos o cuál no vuelve a aparecer.
- **Desempate (Tie-break):** Si dos páginas empatan porque ninguna volverá a usarse en el futuro, aplica un criterio determinista: elige como víctima a la que tenga el **ID de página de menor valor numérico**.

Resultado simular_lru(int N, int secuencia[], int tam)

- **Qué hace:** Simula el algoritmo **LRU (Least Recently Used)**, que reemplaza la página que no ha sido utilizada durante el mayor período de tiempo.
- **Lógica clave:** Utiliza un arreglo auxiliar llamado `ultimo_acceso[N]`. Cada vez que una página se lee (ya sea un acierto/*hit* o un nuevo ingreso), se registra el número de paso (step) actual en este arreglo. Al ocurrir un fallo, busca en `ultimo_acceso` el valor más pequeño (el tiempo más antiguo) para decidir la víctima.
- **Desempate (Tie-break):** En caso de un empate teórico estricto en los tiempos de acceso, desempata expulsando la página con el **ID numérico más bajo**.

3. Demostración y Punto de Entrada

void demostrar_anomalia_belady()

- **Qué hace:** Prueba de forma automatizada la **Anomalía de Belády**, un fenómeno exclusivo de ciertos algoritmos no motivados por pilas (como FIFO) donde tener más memoria empeora el rendimiento.
- **Cómo lo hace:** Ejecuta el simulador FIFO utilizando una secuencia matemática específica de 12 elementos (1, 2, 3, 4, 1, 2, 5...). Primero lo corre asignando 3 marcos (N=3) y luego con 4 marcos (N=4), imprimiendo ambos resultados frente a frente para demostrar que los fallos aumentan en el segundo caso.

```

aldo-reyes@aldo-reyes-QEMU-Virtual-Machine:~/Escritorio/LabsCC7/Lab10_page_replacement$ make run
./page_replacement
--- Simulador de Reemplazo de Paginas (C Version) ---
Ingrese el numero de marcos fisicos (N >= 1): 4

Ejemplo de secuencia recomendada por el lab:
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
Ingrese la secuencia (separada por espacios): 1 2 3 4 5 1 2 3 4

=====
FIFO (N=4)
=====
step  ref  result  frames[0..N-1]  victim
-----
1     1    MISS    [1, _, _, _]    -
2     2    MISS    [1, 2, _, _]    -
3     3    MISS    [1, 2, 3, _]    -
4     4    MISS    [1, 2, 3, 4]    -
5     5    MISS    [5, 2, 3, 4]    1
6     1    MISS    [5, 1, 3, 4]    2
7     2    MISS    [5, 1, 2, 4]    3
8     3    MISS    [5, 1, 2, 3]    4
9     4    MISS    [4, 1, 2, 3]    5

Totals: hits=0 misses=9 | Hit Rate: 0.00%

```

```

=====
MIN / Optimal (N=4)
=====
step  ref  result  frames[0..N-1]  victim
-----
1     1    MISS    [1, _, _, _]    -
2     2    MISS    [1, 2, _, _]    -
3     3    MISS    [1, 2, 3, _]    -
4     4    MISS    [1, 2, 3, 4]    -
5     5    MISS    [1, 2, 3, 5]    4
6     1    HIT     [1, 2, 3, 5]    -
7     2    HIT     [1, 2, 3, 5]    -
8     3    HIT     [1, 2, 3, 5]    -
9     4    MISS    [4, 2, 3, 5]    1

Totals: hits=3 misses=6 | Hit Rate: 33.33%

```

=====				
LRU (N=4)				
=====				
step	ref	result	frames[0..N-1]	victim

1	1	MISS	[1, _, _, _]	-
2	2	MISS	[1, 2, _, _]	-
3	3	MISS	[1, 2, 3, _]	-
4	4	MISS	[1, 2, 3, 4]	-
5	5	MISS	[5, 2, 3, 4]	1
6	1	MISS	[5, 1, 3, 4]	2
7	2	MISS	[5, 1, 2, 4]	3
8	3	MISS	[5, 1, 2, 3]	4
9	4	MISS	[4, 1, 2, 3]	5
Totals: hits=0 misses=9 Hit Rate: 0.00%				

=====

TABLA COMPARATIVA DE RESUMEN

=====

Algoritmo	Hits	Misses	Hit Rate

FIFO	0	9	0.00%
MIN (Optimal)	3	6	33.33%
LRU	0	9	0.00%

=====

=====

BONUS: DEMOSTRACION DE LA ANOMALIA DE BELADY

=====

La anomalia de Belady demuestra que, en FIFO, aumentar el numero de marcos (N) puede INCREMENTAR los fallos.

Secuencia utilizada: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Resultados obtenidos con FIFO:

-> Para N = 3 marcos: Misses = 9

-> Para N = 4 marcos: Misses = 10 (¡Aumento!)

=====